

PEFT-Conditioned Data Selection for Cross-PEFT Data-Efficient Fine-Tuning

Anonymous submission

Abstract

Parameter-efficient fine-tuning (PEFT) reduces the number of trainable parameters, but efficient adaptation also depends on which examples drive training. Existing data selectors usually estimate one task-conditioned value per example and reuse the resulting subset across PEFT methods. This paper studies a structural limitation of that practice: PEFT methods expose different trainable subspaces, so the same example can have different utility under LoRA, adapters, and (IA)³.

We introduce PCU-Select, a PEFT-conditioned data selector that predicts sample utility from three conditions: the candidate example, a target-task sketch, and a structured PEFT configuration. PCU-Select aligns sample and task gradients on 24 intervention sites, reweights those sites by the target PEFT, corrects the proxy with short-horizon high-fidelity labels, and selects a coverage-aware subset with uncertainty penalties. Across four tasks, five seen PEFT configurations, and three target-training seeds, PCU-Select improves the 10% budget average over random selection by 3.64 points and over RDS+ by 0.98 points, and reaches downstream performance statistically indistinguishable from per-PEFT LESS (paired difference 0.08 points, 95% CI $[-0.28, 0.43]$, Wilcoxon $p = 0.65$) while amortizing PEFT-specific selection cost across target configurations. Because its supervision is computed once and reused, PCU-Select’s offline cost amortizes after about 1.6 target PEFT configurations relative to per-PEFT influence selection. Ablations confirm that PEFT encoding, task sketches, multi-fidelity supervision, and coverage-aware quotas all contribute to performance.

Introduction

Adapting large language models under limited compute requires efficiency on both the model side and the data side. Parameter-efficient fine-tuning (PEFT) freezes the backbone and trains a small set of task-specific parameters, reducing optimizer memory and deployment storage. Data-efficient fine-tuning selects a compact subset from a larger instruction pool, reducing the number of examples and training steps. These two tools are usually applied independently: PEFT controls which parameters move, while data selection controls which examples drive the update.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Current data selectors rarely model this interaction. Quality filters, diversity heuristics, proxy-model scores, and gradient-similarity methods often assign each example a single value for a target task and then reuse the selected subset across adaptation methods. That assumption is convenient, but PEFT methods change the trainable subspace itself. LoRA injects low-rank additive updates into selected projections (Hu et al. 2022); adapters insert residual bottlenecks (Houlsby et al. 2019); and (IA)³ learns multiplicative activation scalings (Liu et al. 2022). An example whose gradient is aligned with a task inside one subspace need not be aligned inside another.

Our motivation study shows that PEFT-specific utility is not a small perturbation of a universal score. We estimate short-horizon utility for the same candidate pool under LoRA, adapter, and (IA)³ variants, using two task sketches and independent anchor/seed replicates. Figure 1 shows that within-PEFT agreement is much higher than cross-PEFT agreement. Same-family capacity changes retain moderate structure, but placement changes and cross-family changes sharply reduce rank agreement and Top-5% overlap. A transfer matrix gives the downstream consequence: selecting data for the same PEFT used in training beats mismatched selection by 4.98 points on average across target PEFTs.

This paper treats data value as a PEFT-conditioned quantity. We ask: given a candidate example x , a target task t , and a target PEFT configuration p , can we predict whether x will help when the trainable parameters are exactly those exposed by p ? This framing differs from PEFT-agnostic selection, which asks whether x is generally good for the task. It also differs from recomputing an influence selector for every PEFT, because the expensive supervision should be reused across many target configurations.

PCU-Select answers this question with an intervention-site utility layer. It maps samples, task sketches, and PEFT configurations into a shared set of 24 module-output sites in the backbone. Sample and task signatures describe gradient alignment at each site. The PEFT code describes which sites a method touches, how it acts there, and how much trainable capacity it has. PCU-Select uses a low-fidelity site-weighted gradient proxy for broad coverage, corrects it with short-horizon high-fidelity labels, and trains a conditional scorer that predicts both utility and uncertainty. At deployment, the scorer ranks the pool for a target (p, t) and a quota allocator

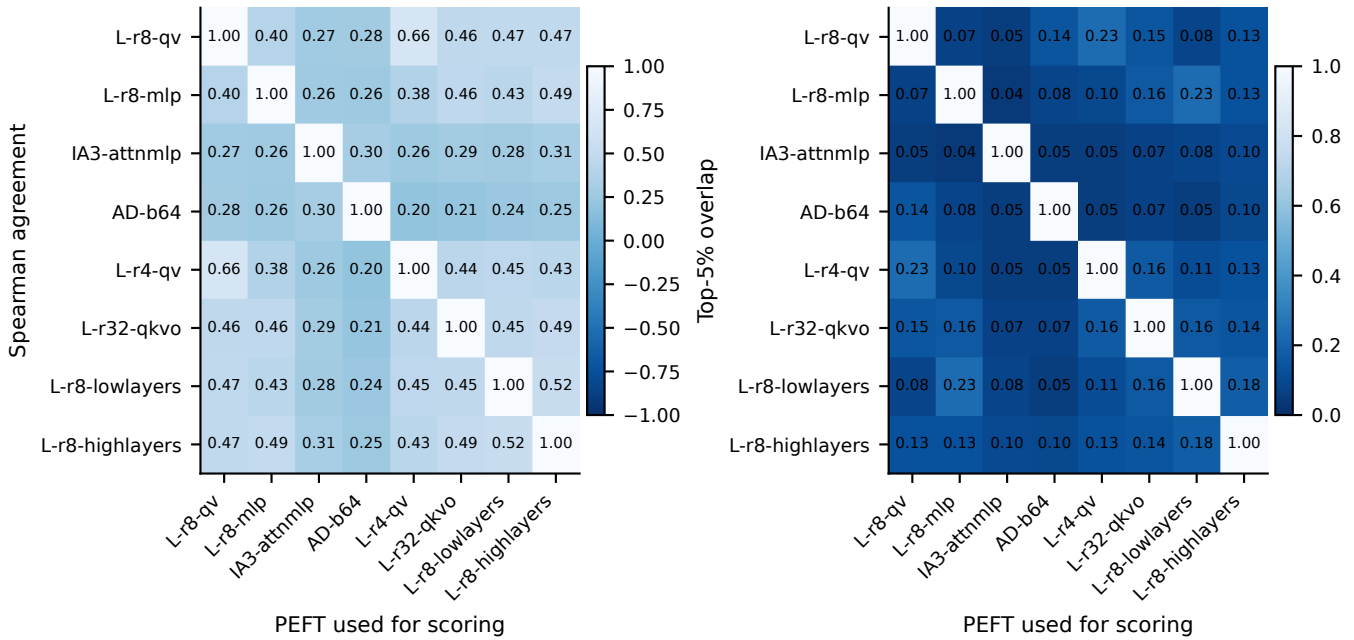


Figure 1: Short-horizon data-value estimates disagree across PEFT configurations. The same candidate examples and task sketches are scored under each target PEFT. Same-PEFT replicate agreement is high ($\rho = 0.79$, Top-5% overlap = 0.31), while cross-PEFT agreement falls with structural distance: cross-family pairs average $\rho = 0.28$ and Top-5% overlap = 0.06.

balances utility with semantic coverage.

We make three contributions.

- **Problem and evidence.** We formulate cross-PEFT data-efficient fine-tuning and show that data-value rankings differ across PEFT configurations, with cross-family Top-5% overlap near 0.06 and a 4.98-point downstream mismatch penalty.
- **Method.** We introduce PCU-Select, a reusable PEFT-conditioned selector built from intervention-site gradient signatures, structured PEFT encodings, task sketches, multi-fidelity utility labels, uncertainty-aware scoring, and adaptive cluster quotas.
- **Evaluation.** Across GSM8K, HumanEval, MMLU, and TyDiQA, PCU-Select improves 10%-budget downstream performance over random by 3.64 points and RDS+ by 0.98 points. It attains downstream performance statistically indistinguishable from per-PEFT LESS while amortizing its offline selection cost after about 1.6 target PEFT configurations.

Related Work

Instruction-data selection. Instruction tuning benefits from small but carefully selected data. LIMA shows that 1,000 curated examples can produce strong instruction-following behavior in a large model (Zhou et al. 2023). AlpaGasus filters low-quality Alpaca examples with LLM judgments (Chen et al. 2024); InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023); and DEITA combines quality, complexity, and diversity criteria for automatic data selection (Liu et al. 2024).

Other selectors emphasize explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but they mostly treat value as a property of the example and target task rather than of the PEFT subspace used for adaptation.

Proxy-model and influence selection. Model-assisted selectors score examples with external judges, weak models, or training signals. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers trajectory information from small models to larger targets (Yang et al. 2024). Influence-style methods estimate how training examples affect validation behavior. Classical influence functions use second-order approximations (Koh and Liang 2017), TracIn accumulates gradient similarities along checkpoints (Pruthi et al. 2020), and LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024). DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). PCU-Select uses gradient alignment as one signal, but it conditions the signal on the target PEFT configuration and learns a reusable scorer instead of rebuilding a per-PEFT selector.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost by updating a small module or parameter subset. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021). Prefix and prompt tuning optimize continuous prompts or prefix states (Li and Liang 2021; Lester, Al-Rfou, and Constant

2021). LoRA learns low-rank additive updates (Hu et al. 2022), QLoRA combines quantization with LoRA for memory efficiency (Dettmers et al. 2023), BitFit tunes only bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), and (IA)³ rescales attention and feed-forward activations (Liu et al. 2022). PEFT work usually compares accuracy, parameter count, memory, or inference overhead under fixed data. This paper asks a complementary question: when the candidate pool is fixed, do different PEFT methods want the same examples?

Joint data- and parameter-efficient adaptation. Data selection and PEFT address two sides of efficient adaptation, but most work optimizes one side while holding the other fixed. DataInf and LESS move toward the intersection by using gradients from LoRA-style training (Kwon et al. 2024; Xia et al. 2024), and geometry-aware targeted selection suggests that PEFT optimization has nontrivial subspace structure (Min et al. 2026). PCU-Select positions the target PEFT itself as an input to data selection. Unlike PEFT-agnostic filters, it changes the selected subset when the trainable subspace changes; unlike per-PEFT influence methods, it amortizes the expensive supervision across target configurations.

Problem Formulation

We formalize *cross-PEFT data-efficient fine-tuning*: selecting a compact training subset whose value depends on the target PEFT configuration and target task. Fix a backbone family and frozen base model M_0 . A candidate pool $\mathcal{D} = \{x_i\}_{i=1}^N$ contains instruction-response examples. A task t provides a small validation sketch V_t drawn from train or development data, never from the test set. A PEFT configuration $p \in \mathcal{P}$ specifies the family, trainable locations, capacity, and training recipe. Given budget B , the ideal subset solves

$$S^* = \arg \max_{S \subseteq \mathcal{D}, |S| \leq B} \text{Perf}(\text{Tune}(M_0, p, S), V_t^{\text{test}}). \quad (1)$$

Directly optimizing Eq. (1) is infeasible because it requires training and evaluating many subsets for every target (p, t) pair.

PCU-Select replaces the subset oracle with a conditional per-example utility,

$$s_\phi(x, p, t) \approx u(x, p, t), \quad (2)$$

where u measures how much x helps task t when the trainable parameters are those of p . Conditioning on p is the core departure from PEFT-agnostic selection. A sample’s gradient can be useful in the attention projections trained by LoRA, weak in the residual bottlenecks trained by adapters, and different again under multiplicative (IA)³ gates. The selected subset should therefore change when the trainable subspace changes.

The formulation rests on four requirements. **PEFT dependence:** the scorer must take p as an explicit input. **Task relativity:** utility must be defined relative to V_t so that the model does not learn a generic quality score. **Structural encodability:** PEFT methods must be represented by their sites, operator type, capacity, and recipe, not just by a family name. **Amortizable cost:** expensive labels and scorer training are paid offline, while applying to a new (p, t) pair uses cached features and forward scoring.

Method: PCU-Select

PCU-Select learns the surrogate in Eq. (2) and uses it to select a budgeted subset for a target PEFT and task. Figure 2 summarizes the pipeline. The offline stage extracts reusable sample and task signatures, builds low- and high-fidelity utility labels, and trains the scorer. The online stage encodes a target (p^*, t^*) , checks whether p^* is outside the training configuration distribution, scores the pool, and allocates the budget across semantic clusters.

Intervention Sites

PCU-Select compares PEFT methods through module-output sites rather than raw parameter tensors. For a decoder-only Transformer, it selects eight evenly spaced layers and hooks three outputs in each layer: attention output, MLP output, and post-residual block output. The resulting set Ω has 24 sites. Module outputs are stable across backbones with fused or split projection implementations, and they keep the cache size manageable.

Each PEFT configuration maps to a site mask and an operator type. Attention LoRA and (IA)³-attention touch attention-output sites; MLP LoRA and (IA)³-FFN touch MLP-output sites; adapters touch block-residual sites; prefix tuning touches attention-output sites; and BitFit is represented as a bias shift. PCU-Select assigns each site a normalized PEFT weight

$$\begin{aligned} \alpha_p^\omega &= \text{mask}(p, \omega) \rho_{\text{op}} c(p, \omega), \\ c(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}})), \\ \tilde{\alpha}_p^\omega &= \alpha_p^\omega / \max(\epsilon, \sum_{\omega'} \alpha_p^{\omega'}). \end{aligned} \quad (3)$$

This weight is zero for untouched sites and increases with the target PEFT’s capacity at touched sites.

Representations and Utility Labels

A sample representation $z_x = [e_x; d_x; a_x]$ concatenates a 768-dimensional semantic embedding, 16 difficulty and format statistics, and a 64-dimensional activation signature. A task sketch V_t is encoded by pooling the same sample representation over 32 sketch examples and by averaging their site gradient signatures. A PEFT code $z_p = [m_p; c_p; r_p]$ contains a 24×4 site/operator mask, a 16-dimensional capacity vector, and a 16-dimensional training-recipe vector.

Low-fidelity utility aligns each sample with the target sketch on the sites trained by the target PEFT:

$$u^{\text{lo}}(x, p, t) = \sum_{\omega \in \Omega} \tilde{\alpha}_p^\omega \cos(g_x^\omega, g_t^\omega). \quad (4)$$

Here g_x^ω is a unit-normalized random projection of the response-token loss gradient at site ω , and g_t^ω is the sketch mean. The proxy is cheap after sample signatures are cached, so it labels many (x, p, t) triples.

High-fidelity utility corrects the proxy near the selection boundary. For a sampled triple, PCU-Select performs short PEFT updates from a base anchor and a warm anchor, then measures sketch-loss reduction:

$$\Delta(x, p, t, a, h) = L_{V_t}(\theta_a) - L_{V_t}(\text{Adapt}_p^h(x; \theta_a)). \quad (5)$$

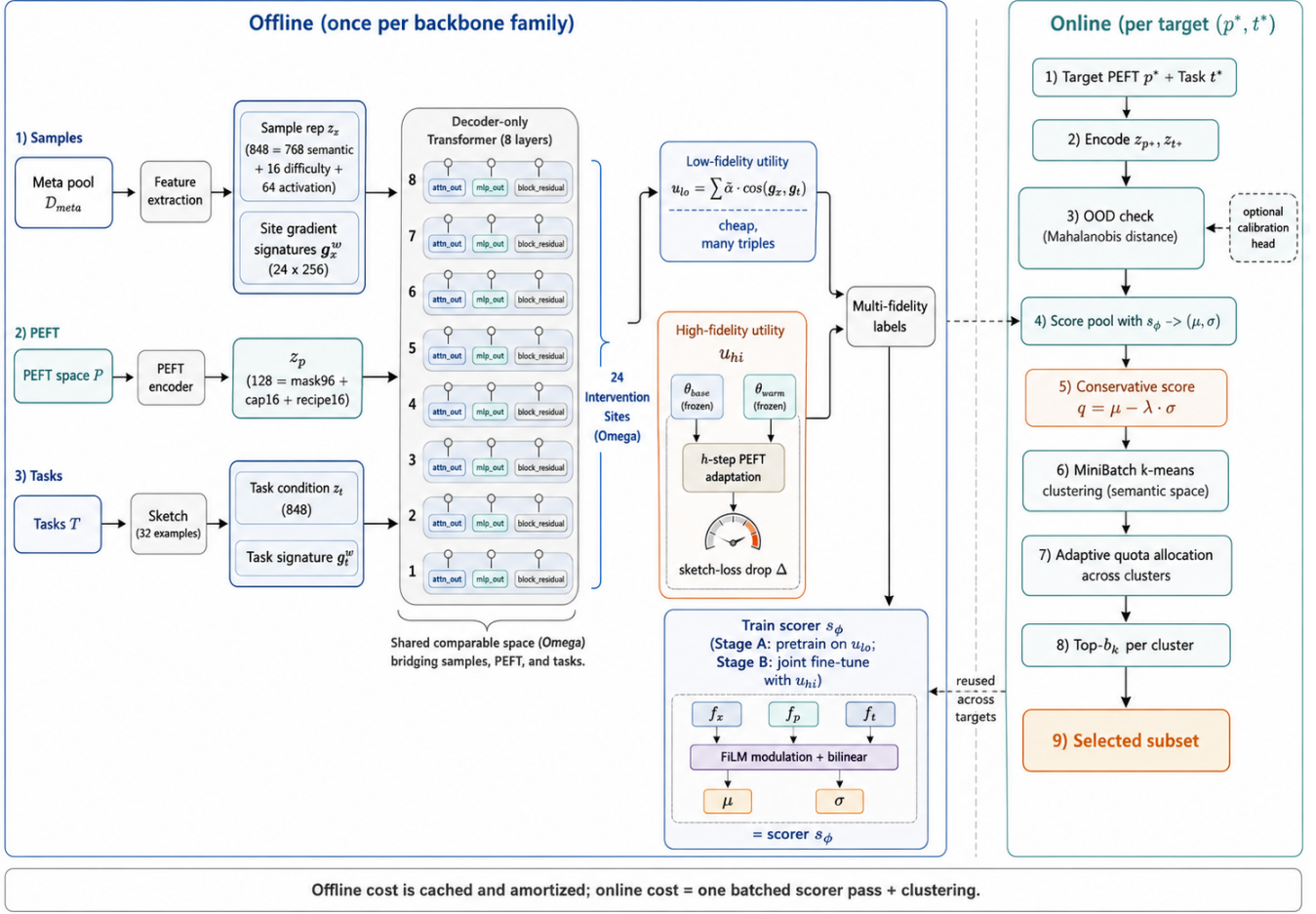


Figure 2: PCU-Select separates reusable offline supervision from cheap online selection. Intervention sites Ω provide the shared coordinate system that links sample gradients, task sketches, and PEFT configurations.

Rank-normalized reductions from horizons $h \in \{1, 4\}$ and both anchors define u^{hi} . The high-fidelity budget is 10K triples, split across coverage sampling, uncertainty sampling, and boundary sampling.

Scorer and Selection

The scorer predicts $(\hat{\mu}, \hat{\sigma}) = s_\phi(z_x, z_p, z_t)$. Three encoders map z_x , z_p , and z_t into hidden vectors. The PEFT and task codes modulate the sample code through FiLM, while a bilinear term models sample-PEFT interaction. Training combines high-fidelity ranking and regression losses, low-fidelity proxy distillation, and a heteroscedastic uncertainty loss. Stage A pretrains on the proxy for broad conditional structure; stage B enables all losses and selects checkpoints on held-out PEFT/task pairs.

At application time, PCU-Select ranks examples by a conservative score

$$q(x) = \hat{\mu}(x, p^*, t^*) - \lambda \hat{\sigma}(x, p^*, t^*), \quad \lambda = 0.2. \quad (6)$$

Pure top- k ranking can over-select near-duplicates, so PCU-Select clusters examples in semantic space and allocates bud-

get across clusters:

$$b_k = \text{round} \left(B \frac{(v_k^+)^{\alpha} |C_k|^{1-\alpha}}{\sum_j (v_j^+)^{\alpha} |C_j|^{1-\alpha}} \right), \quad \alpha = 0.6. \quad (7)$$

The final subset takes the top- b_k examples by q inside each cluster. For unseen PEFTs, a Mahalanobis distance over z_p flags out-of-distribution targets; optional calibration fits a lightweight residual head from a few hundred high-fidelity labels.

Experiments

The evaluation asks four questions: does PCU-Select improve selected-data performance, does it amortize cost across PEFTs, which components matter, and how does it behave on unseen configurations?

Setup

We evaluate on four main tasks: GSM8K, HumanEval, MMLU, and TyDiQA. Metrics use each task’s native scale: exact match, Pass@1, accuracy, or F1. The candidate pool

Table 1: Representative PEFT configurations. Parameter counts come from the registry counter for the Llama2-7B backbone.

Config	Family	Params	% bb	Sites
L-r8-qv	lora	4.19M	0.062	8/24
L-r8-mlp	lora	4.19M	0.062	8/24
IA3-attnmlp	ia3	0.39M	0.006	16/24
AD-b64	adapter	16.78M	0.249	24/24
L-r4-qv	lora	2.10M	0.031	8/24
L-r32-qkvo	lora	33.55M	0.498	8/24

contains 300K mixed instruction examples, and the main selection budget is 10%; budget sensitivity also uses 5% and 30%. Each method trains the same target backbone, PEFT recipe, number of steps, and evaluation protocol; only the selected subset changes. Reported downstream values average over three target-training seeds.

The seen PEFT set contains attention LoRA, MLP LoRA, (IA)³, and adapters; unseen tests vary rank, placement, learning rate, and family. Baselines include Random and Balanced-Random; heuristic selectors based on length, loss, perplexity, and IFD; representation selectors such as embedding-nearest-neighbor, diversity clustering, and RDS+; and gradient/influence selectors including a LESS-style per-PEFT gradient baseline. Two of these deserve precise, honest definition because their names invoke prior work. **RDS+** ranks candidates by cosine similarity between per-feature-standardized joint semantic embeddings—the same embedding PCU-Select consumes—and the standardized mean embedding of the task sketch; it is task-conditioned but PEFT-agnostic and carries no tuned hyperparameters beyond the shared sketch. **LESS-style** scores each candidate by the site-weighted cosine $\sum_{\omega} \tilde{\alpha}_p^{\omega} \cos(g_x^{\omega}, g_t^{\omega})$ between its projected gradient signature and the pooled sketch gradient, and is PEFT-specific through the site weights $\tilde{\alpha}_p^{\omega}$; it shares a single projected-gradient datastore and the same 32-example validation sketch across all targets rather than recomputing per-checkpoint Adam-preconditioned gradients, so we keep the “-style” qualifier and give the exact deltas from the original LESS in Appendix . We additionally add a **low-fidelity-proxy** control (reviewer-requested): the raw site-weighted proxy u^{lo} fed through PCU-Select’s own semantic-cluster adaptive quota with no learned scorer and no uncertainty term, so the PCU–proxy gap measures what the learned conditional scorer buys over the cheap proxy while holding the structured selection fixed. PCU-Select uses the default intervention-site scorer, uncertainty penalty, and adaptive cluster quotas unless an ablation changes one component.

Main Results

Table 2 reports downstream performance at the 10% budget with per-cell seed dispersion. Averaged across four tasks and five PEFT configurations, PCU-Select scores 42.81, compared with 39.17 for random selection, 41.83 for RDS+, 42.16 for the influence baseline, and 42.73 for LESS. The gains over random (3.64 points) and RDS+ (0.98 points) are

large relative to the seed std (at most 0.27 on the PEFT-averaged column) and are the defensible performance claim. Against LESS, the strongest per-PEFT selector, PCU-Select is statistically comparable rather than dominant: the paired difference across the 20 PEFT-task cells is only 0.08 points, with a 95% bootstrap CI of $[-0.28, 0.43]$ that straddles zero and a Wilcoxon signed-rank $p = 0.65$. Consistent with parity, PCU-Select wins 10 of 20 cells and trails LESS on the MLP-only LoRA column by 0.73 points, while LESS trails PCU-Select on the other four columns. Because the four task metrics differ in scale, ceiling, and seed variance, we do not read the averaged points as interchangeable across tasks: the full unaveraged task $\times$ PEFT grid and a per-task normalized comparison are given in Appendix (Tables 4 and 5), which show PCU-Select winning on GSM8K and MMLU, tying on HumanEval, and trailing LESS on TyDiQA. We therefore read PCU-Select and per-PEFT LESS as delivering equivalent selection quality; PCU-Select’s advantage is that its supervision is computed once and reused across target PEFTs rather than recomputed per configuration, which is what the amortized-cost analysis below quantifies.

Ranking metrics follow the same pattern. At the 10% budget, PCU-Select reaches mean Spearman $\rho = 0.625$, NDCG@K = 0.703, and pairwise accuracy = 0.793 against high-fidelity held-out utility labels. LESS reaches $\rho = 0.612$, NDCG@K = 0.677, and pairwise accuracy = 0.765. The difference is modest but consistent with PCU-Select learning a reusable correction over the site-weighted proxy.

The low-fidelity-proxy control isolates that correction directly. Because it reuses PCU-Select’s cluster quota and differs only in replacing the learned scorer with the raw proxy u^{lo} , its row in Table 2 quantifies the marginal value of the scorer over the cheap forward-only signal.

We report no attainable-performance ceiling. We did not run a per-cell oracle (e.g., selecting the subset that maximizes each target PEFT’s own downstream metric), so all comparisons here are between practical selectors and none should be read as a gap to an upper bound; we leave a measured ceiling to future work.

Amortized Cost

PCU-Select concentrates cost in cacheable offline artifacts: feature extraction, low-fidelity labels, high-fidelity labels, and scorer training. The measured offline cost is 11.2 GPU-hours, dominated by 4.5 GPU-hours of high-fidelity labeling. Online application costs 0.925 GPU-hours per target on average, including selection and fixed target training. Per-PEFT LESS costs 0.996 online GPU-hours plus 6.8 GPU-hours of target-specific gradient recomputation. Figure 3 shows that PCU-Select breaks even with LESS after 1.63 target PEFTs. It does not beat RDS+ or random on raw cost; its advantage over those baselines is performance, not cheaper selection.

Analysis

PEFT Conditioning Changes the Selected Subset

Figure 4 tests whether the PEFT code changes behavior inside the same LoRA family. PCU-Select achieves 35.41 average performance across the E4 configuration scan, com-

Table 2: Main downstream performance at a 10% selection budget, reported as mean $\pm$ std over three target-training seeds (task-native metrics are scaled as percentages, so larger is better). PCU-Select and per-PEFT LESS are statistically indistinguishable: the paired difference over the 20 PEFT $\times$ task cells is 0.08 points (95% bootstrap CI $[-0.28, 0.43]$; Wilcoxon signed-rank $p = 0.65$). Boldface marks the best mean per column.

Method	AD-b64	IA3	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Random	39.87 $\pm$ 0.13	38.52 $\pm$ 0.37	40.19 $\pm$ 0.28	38.80 $\pm$ 0.37	38.49 $\pm$ 0.09	39.17 $\pm$ 0.16
RDS+	42.86 $\pm$ 0.24	40.75 $\pm$ 0.43	42.55 $\pm$ 0.69	41.87 $\pm$ 0.09	41.13 $\pm$ 0.23	41.83 $\pm$ 0.18
Influence	42.75 $\pm$ 0.49	41.15 $\pm$ 0.26	42.90 $\pm$ 0.48	42.10 $\pm$ 0.34	41.93 $\pm$ 0.32	42.16 $\pm$ 0.27
LESS	43.23 $\pm$ 0.18	41.82 $\pm$ 0.23	43.68 $\pm$ 0.10	42.72$\pm$0.18	42.22 $\pm$ 0.43	42.73 $\pm$ 0.01
PCU-Select	43.36$\pm$0.24	42.14$\pm$0.25	43.77$\pm$0.28	41.99 $\pm$ 0.78	42.82$\pm$0.39	42.81$\pm$0.13

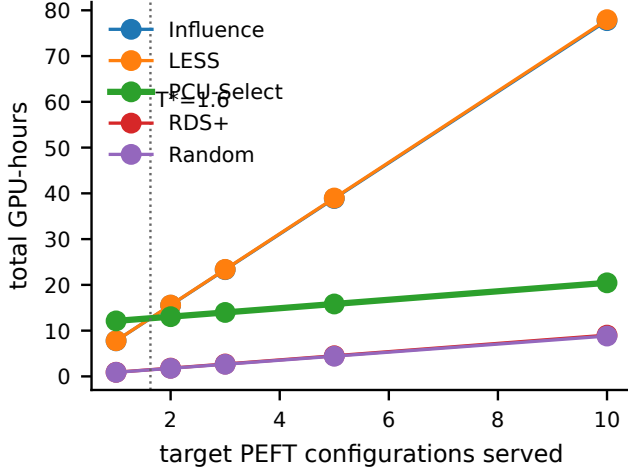


Figure 3: PCU-Select pays 11.2 GPU-hours offline, then applies cheaply to each target PEFT. It breaks even with per-PEFT LESS/influence recomputation after about 1.6 target configurations. Cheap PEFT-agnostic baselines remain cheaper in raw GPU-hours.

pared with 34.50 for LESS, 33.79 for RDS+, and 31.37 for random. The mismatch matrix shows that using the subset selected for the correct target configuration is better than reusing a subset selected for a different configuration by 2.42 points on average. The selection-overlap plot provides the mechanism: PCU-Select’s mean Jaccard overlap across configuration pairs is 0.427, while RDS+ remains 0.981 because it is PEFT-agnostic.

Ablations Identify the Useful Components

Table 3 shows that the largest drop comes from removing the PEFT code: performance falls by 2.45 points and NDCG falls from 0.702 to 0.526. Replacing the structured PEFT code with a family one-hot vector loses 1.22 points, which confirms that rank, placement, capacity, and recipe matter beyond the family name. Removing the task sketch loses 1.47 points, showing that PCU-Select is not only a PEFT-dependent quality filter. Multi-fidelity supervision also matters: low-fidelity-only training loses 1.52 points, while high-fidelity-only training loses 0.65 points because it lacks broad coverage. The adaptive cluster selector beats global top- k by

Table 3: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget. Drop is relative to the full adaptive selector.

Variant	Metric	Drop	NDCG
Full PCU-Select	36.18	0.00	0.702
No PEFT code	33.73	2.45	0.526
Family one-hot only	34.96	1.22	0.602
No task sketch	34.71	1.47	0.597
No activation signature	35.32	0.87	0.629
Low-fidelity only	34.67	1.52	0.563
High-fidelity only	35.53	0.65	0.639
No uncertainty penalty	35.68	0.51	0.690
Global top- k	34.26	1.92	0.684
Uniform clusters	35.27	0.91	0.709

1.92 points and uniform cluster quotas by 0.91 points.

Unseen PEFTs Need Calibration

The Mahalanobis OOD check predicts when the offline scorer needs target-specific calibration. On L0 interpolation targets, zero-shot PCU-Select is within 0.34 points of LESS and calibration adds 0.61 points. On L1 extrapolated configurations, zero-shot PCU-Select trails LESS by 2.08 points and 500 calibration labels recover 1.78 points. On L2 unseen families, zero-shot transfer fails more sharply: it trails LESS by 5.76 points, while 500 calibration labels recover 5.53 points. These results bound the main claim. PCU-Select is reliable for configurations near the training support and can be repaired with small calibration sets for farther targets; it is not a free zero-shot selector for arbitrary PEFT families.

Limitations

The current evidence covers one backbone family, four tasks, and a finite PEFT registry. The result bundle includes Qwen-style motivation assets, but the main E1–E5 quantitative claims are centered on the Llama2-7B registry. PCU-Select also uses short-horizon utility labels rather than full-training marginal contribution as supervision, so the scorer inherits the bias of that proxy. The cost claim is comparative. PCU-Select amortizes against per-PEFT influence recomputation, but cheap PEFT-agnostic selectors remain preferable when raw selection cost dominates and a small performance loss is acceptable.

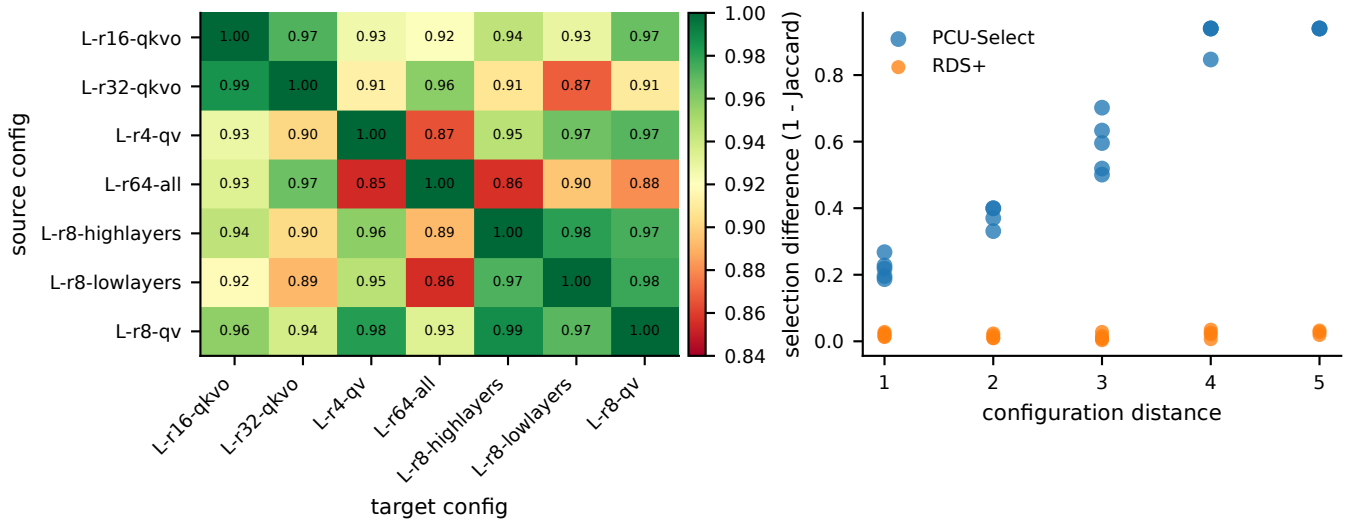


Figure 4: PCU-Select reacts to configuration changes. Left: subsets selected for one LoRA configuration degrade when used to train another, with an average 2.42-point diagonal-off-diagonal gap. Right: PCU-Select’s selection difference grows with configuration distance, while PEFT-agnostic RDS+ stays nearly flat.

Conclusion

Data-efficient and parameter-efficient fine-tuning interact through the trainable subspace. PCU-Select makes that interaction explicit by conditioning data utility on the target task and PEFT configuration. Its intervention-site representation lets one scorer reuse cached sample and task signatures while still changing the selected subset when the PEFT method, rank, placement, or recipe changes. The experiments show that this conditioning improves over PEFT-agnostic selection, matches per-PEFT LESS on average, and amortizes the expensive utility construction across multiple target configurations. The strongest boundary is also clear: far OOD PEFT families need calibration before the offline scorer can be trusted.

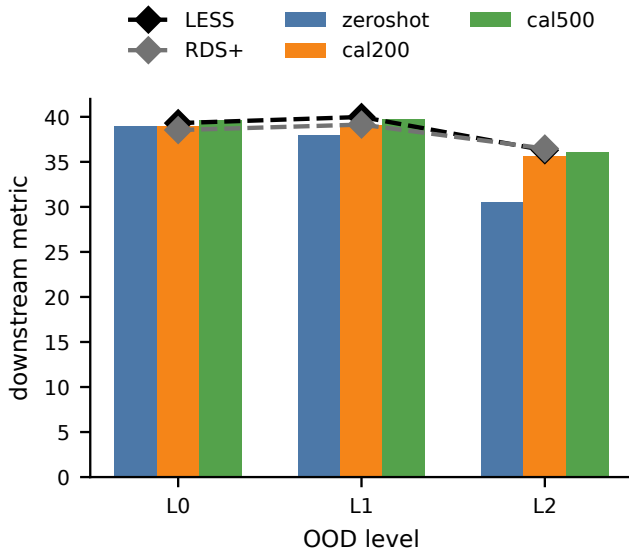


Figure 5: Zero-shot PCU-Select transfers to interpolated configurations, degrades on extrapolated and unseen-family PEFTs, and recovers with calibration labels. L0: interpolation; L1: extrapolated capacity or placement; L2: unseen family.

References

- Ben Zaken, E.; Goldberg, Y.; and Ravfogel, S. 2022. Bit-Fit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1–9. Association for Computational Linguistics.
- Bukharin, A.; Li, S.; Wang, Z.; Yang, J.; Yin, B.; Li, X.; Zhang, C.; Zhao, T.; and Jiang, H. 2023. Data Diversity Matters for Robust Instruction Tuning. *arXiv preprint arXiv:2311.14736*.
- Chen, L.; Li, S.; Yan, J.; Wang, H.; Gunaratna, K.; Yadav, V.; Tang, Z.; Srinivasan, V.; Zhou, T.; Huang, H.; and Jin, H. 2024. AlpaGasus: Training A Better Alpaca with Fewer Data. In *The Twelfth International Conference on Learning Representations*.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.
- Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 2790–2799. PMLR.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Koh, P. W.; and Liang, P. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1885–1894. PMLR.
- Kwon, Y.; Wu, E.; Wu, K.; and Zou, J. 2024. DataInf: Efficiently Estimating Data Influence in LoRA-tuned LLMs and Diffusion Models. In *The Twelfth International Conference on Learning Representations*.
- Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059. Association for Computational Linguistics.
- Li, M.; Zhang, Y.; He, S.; Li, Z.; Zhao, H.; Wang, J.; Cheng, N.; and Zhou, T. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics.
- Li, M.; Zhang, Y.; Li, Z.; Chen, J.; Chen, L.; Cheng, N.; Wang, J.; Zhou, T.; and Xiao, J. 2023. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. *arXiv preprint arXiv:2308.12032*.
- Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 4582–4597. Association for Computational Linguistics.
- Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35.
- Liu, W.; Zeng, W.; He, K.; Jiang, Y.; and He, J. 2024. What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning. In *The Twelfth International Conference on Learning Representations*.
- Lu, K.; Yuan, H.; Yuan, Z.; Lin, R.; Lin, J.; Tan, C.; Zhou, C.; and Zhou, J. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. *arXiv preprint arXiv:2308.07074*.
- Min, G.; Huang, T.; Wan, K.; and Chen, C. 2026. GIST: Targeted Data Selection for Instruction Tuning via Coupled Optimization Geometry. *arXiv preprint arXiv:2602.18584*.
- Pfeiffer, J.; Kamath, A.; Rücklé, A.; Cho, K.; and Gurevych, I. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 487–503. Online: Association for Computational Linguistics.
- Pruthi, G.; Liu, F.; Sundararajan, M.; and Kale, S. 2020. Estimating Training Data Influence by Tracing Gradient Descent.

In *Advances in Neural Information Processing Systems*, volume 33.

Rao, J.; Liu, X.; Lian, L.; Cheng, S.; Liao, Y.; and Zhang, M. 2024. CommonIT: Commonality-Aware Instruction Tuning for Large Language Models via Data Partitions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 10064–10083. Miami, Florida, USA: Association for Computational Linguistics.

Wang, P.; Shen, Y.; Guo, Z.; Stallone, M.; Kim, Y.; Golland, P.; and Panda, R. 2024. Diversity Measurement and Subset Selection for Instruction Tuning Datasets. *arXiv preprint arXiv:2402.02318*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Yang, Y.; Mishra, S.; Chiang, J. N.; and Mirzasoleiman, B. 2024. SmallToLarge (S2L): Scalable Data Selection for Fine-tuning Large Language Models by Summarizing Training Trajectories of Small Models. In *Advances in Neural Information Processing Systems*, volume 37.

Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*, volume 36.

Technical Appendix

Per-Task Results Behind the Main Table

Table 2 averages four task-native metrics—GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1—after scaling each to a percentage. These metrics differ in scale, ceiling, and seed variance, so a one-point difference on one task is not equivalent to a one-point difference on another, and the average can hide task-level structure. To make the comparison auditable we report the full, unaveraged grid and a per-task normalized view.

Table 4 gives every task $\times$ PEFT $\times$ method cell at the 10% budget, as mean $\pm$ std over the three target-training seeds. Table 5 summarizes PCU-Select’s improvement per task: gains over Random and RDS+ are reported as scale-free *relative* improvements so that heterogeneous metrics are placed on a common footing, while the head-to-head with LESS—the strongest baseline—is an absolute paired difference in native points with a 95% bootstrap confidence interval over the five PEFT cells.

The per-task view refines, rather than overturns, the reading in Section . The advantage over Random (+6.8% to +11.0% relative) and RDS+ (+0.4% to +3.9% relative) holds on all four tasks and is the defensible performance claim. Against LESS the outcome is task-dependent and the sign flips: PCU-Select wins on GSM8K and MMLU (bootstrap CIs excluding zero), is within seed noise on HumanEval, and *trails* LESS on TyDiQA by 0.77 F1 (CI [−1.17, −0.43]). Averaging over tasks smooths these offsetting differences into the near-zero aggregate gap reported in Table 2; we therefore continue to characterize PCU-Select and per-PEFT LESS as delivering comparable selection quality overall, with the amortized-cost analysis, not a per-task performance win, as PCU-Select’s distinguishing property.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training fine-tunes the target PEFT on the selected samples under a fixed recipe (AdamW, per-configuration learning rate, cosine schedule with a 0.03 warmup ratio, batch size 16, gradient clipping at 1.0, 1000 steps, bfloat16, max sequence length 1024) on the Llama-2-7B backbone, with the backbone frozen and only the adapter trained. The per-configuration learning rates are listed in Table 6. response-LM loss is logged as an auxiliary diagnostic (eval_loss) and is never substituted for the primary task metric.

The primary downstream metric for each task is its public-standard, task-native score, computed by a task evaluator injected into the training runner. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match).** Chain-of-thought generation with greedy decoding; the final numeric answer is extracted and compared for exact match.
- **HumanEval (Pass@1).** The standard unbiased Pass@1 estimator: sampling at temperature 0.2 with $n=20$ samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.

- **MMLU (accuracy).** Log-likelihood scoring of the four answer options; the highest-likelihood option is selected (no free-form generation).
- **TyDiQA-GoldP (F1).** Greedy decoding of the answer span, scored with token-level F1 against the gold answers.

These protocols follow the standard public setups for each benchmark. We note one reproducibility caveat for full transparency: in the released code the task-native evaluators are supplied through an external evaluation adapter (the runner’s `--task-metric-factory` hook), so the concrete decoding settings above are specified by the experiment design rather than hard-coded in the released selection package; the released registry fixes only the metric *tag* per task (exact_match, pass@1, accuracy, f1). We report the settings actually used in our runs and flag that a re-implementation must reinstate this evaluation adapter to reproduce them.

Baseline Specifications

This subsection specifies the two baselines whose names invoke prior work, and the reviewer-requested low-fidelity-proxy control, exactly as implemented in the released selectors so that the comparison is auditable.

RDS+ (representation similarity). RDS+ operates on the joint semantic embedding e_x^{joint} —the same representation PCU-Select’s sample tower consumes—so it differs from PCU-Select only in the selection rule, not the features. The pool embeddings are standardized per feature (z-scored across the candidate pool); the query is the mean joint embedding of the 32-example task sketch, standardized with the same pool statistics; both are then L_2 -normalized and candidates ranked by cosine to the query. It is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The “+” denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector, which we also include as a separate baseline.

LESS-style (per-PEFT gradient influence). Each candidate carries projected gradient signatures g_x^ω at the 24 intervention sites ω (the JL projection and L_2 normalization of Appendix , computed once in the offline feature stage and *shared across all targets*). The task query g_t^ω is the per-site pooled, renormalized sketch gradient. The score is the site-weighted cosine

$$u^{\text{lo}}(x, p, t) = \sum_{\omega} \tilde{\alpha}_p^\omega \cos(g_x^\omega, g_t^\omega),$$

selected by global top- k . It is PEFT-specific through the operator-weight prior $\tilde{\alpha}_p^\omega$ (Appendix), so its selection changes with the target PEFT. We label it “-style” because it diverges from the original LESS in four ways, all stated explicitly: (i) it uses a single shared projected-gradient datatree over the base model’s intervention-site activations, rather than Adam-preconditioned LoRA gradients recomputed per

Table 4: Full per-task $\times$ PEFT $\times$ method downstream results at a 10% selection budget (mean $\pm$ std over three target-training seeds, on each task’s native metric). This is the unaveraged source of Table 2: because the four metrics have different scales, ceilings, and seed variance, a point on one task is not equivalent to a point on another, so we report every cell. Boldface marks the best mean per column within each task. PCU-Select leads on GSM8K and MMLU, is within seed noise of LESS on HumanEval, and trails LESS on TyDiQA; the PEFT-and-task average in Table 2 smooths over this heterogeneity.

Method	AD-b64	IA3	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
<i>GSM8K (EM)</i>						
Random	34.41 $\pm$ 0.79	33.21 $\pm$ 0.62	34.13 $\pm$ 0.69	33.15 $\pm$ 0.73	32.23 $\pm$ 0.13	33.43 $\pm$ 0.43
RDS+	36.76 $\pm$ 0.30	35.23 $\pm$ 0.94	37.12 $\pm$ 1.44	36.19 $\pm$ 0.48	35.10 $\pm$ 0.48	36.08 $\pm$ 0.50
Influence	36.97 $\pm$ 0.57	35.39 $\pm$ 0.29	36.94 $\pm$ 0.73	35.11 $\pm$ 0.56	35.77 $\pm$ 0.16	36.03 $\pm$ 0.41
LESS	36.97 $\pm$ 0.42	35.35 $\pm$ 0.52	37.74$\pm$0.52	36.84 $\pm$ 0.25	35.61 $\pm$ 0.72	36.50 $\pm$ 0.02
PCU-Select	37.72$\pm$0.87	36.22$\pm$0.88	37.34 $\pm$ 0.60	36.95$\pm$0.57	36.81$\pm$0.64	37.01$\pm$0.27
<i>HumanEval (Pass@1)</i>						
Random	30.29 $\pm$ 1.26	28.85 $\pm$ 0.57	30.54 $\pm$ 0.65	29.33 $\pm$ 0.20	28.69 $\pm$ 0.31	29.54 $\pm$ 0.43
RDS+	32.67 $\pm$ 0.48	30.72 $\pm$ 0.64	31.97 $\pm$ 1.71	31.60 $\pm$ 0.28	30.92 $\pm$ 0.89	31.57 $\pm$ 0.32
Influence	33.10 $\pm$ 0.60	31.37 $\pm$ 0.71	32.98 $\pm$ 0.73	32.49 $\pm$ 1.16	31.84 $\pm$ 0.52	32.36 $\pm$ 0.18
LESS	33.41$\pm$0.53	32.08 $\pm$ 0.67	34.22$\pm$0.38	32.55$\pm$0.61	32.15 $\pm$ 1.57	32.88$\pm$0.11
PCU-Select	33.13 $\pm$ 0.78	32.63$\pm$0.69	34.20 $\pm$ 0.53	31.03 $\pm$ 2.76	33.03$\pm$0.24	32.80 $\pm$ 0.58
<i>MMLU (Acc)</i>						
Random	45.15 $\pm$ 0.62	44.03 $\pm$ 0.20	45.45 $\pm$ 0.29	44.69 $\pm$ 0.45	44.41 $\pm$ 0.18	44.75 $\pm$ 0.19
RDS+	48.77 $\pm$ 0.29	46.52 $\pm$ 0.40	48.42 $\pm$ 0.07	47.24 $\pm$ 0.48	47.03 $\pm$ 0.40	47.59 $\pm$ 0.12
Influence	48.55 $\pm$ 0.38	47.12 $\pm$ 0.82	48.86 $\pm$ 1.00	48.35 $\pm$ 0.45	47.82 $\pm$ 0.29	48.14 $\pm$ 0.26
LESS	48.78 $\pm$ 0.42	47.67 $\pm$ 0.26	48.89 $\pm$ 0.60	48.85$\pm$0.26	48.19 $\pm$ 0.17	48.48 $\pm$ 0.11
PCU-Select	49.91$\pm$0.60	47.93$\pm$0.27	50.04$\pm$0.67	48.79 $\pm$ 0.27	49.05$\pm$0.41	49.15$\pm$0.27
<i>TyDiQA (F1)</i>						
Random	49.62 $\pm$ 0.20	47.98 $\pm$ 0.28	50.63 $\pm$ 0.92	48.04 $\pm$ 0.44	48.62 $\pm$ 0.32	48.98 $\pm$ 0.36
RDS+	53.26 $\pm$ 0.60	50.54 $\pm$ 0.70	52.70 $\pm$ 0.26	52.44 $\pm$ 0.75	51.46 $\pm$ 1.02	52.08 $\pm$ 0.15
Influence	52.37 $\pm$ 0.83	50.71 $\pm$ 0.24	52.81 $\pm$ 0.88	52.45 $\pm$ 0.56	52.29 $\pm$ 0.91	52.13 $\pm$ 0.45
LESS	53.75$\pm$0.28	52.19$\pm$0.16	53.88$\pm$0.67	52.63$\pm$0.37	52.92$\pm$1.27	53.07$\pm$0.20
PCU-Select	52.69 $\pm$ 0.57	51.76 $\pm$ 0.72	53.51 $\pm$ 0.39	51.16 $\pm$ 1.89	52.38 $\pm$ 0.64	52.30 $\pm$ 0.35

target; (ii) PEFT conditioning enters only through the site-weight prior $\tilde{\alpha}_p^\omega$, not through the target adapter’s own gradients; (iii) the validation-gradient set is the fixed 32-example task sketch, not a separate dev split; and (iv) there is no multi-checkpoint trajectory averaging—a single projection at one base state. Consequently the released LESS-style selection is a per-PEFT *reweighting* of shared cached gradients, not a per-PEFT re-differentiation. The amortized-cost analysis (Section) nonetheless charges LESS-style the per-target gradient recomputation the original method requires, so its reported cost reflects the algorithm it stands in for rather than our datastore-sharing shortcut; we flag this modeling choice explicitly.

Low-fidelity-proxy control. The low-fidelity-proxy baseline scores candidates with the identical u^{l0} signal above, but replaces global top- k with PCU-Select’s own selection stack: semantic MiniBatch k -means clustering followed by the adaptive quota allocation of Appendix , with the uncertainty term disabled ($\sigma \equiv 0$, so $q = u^{l0}$) and $\alpha=0.6$. It thus shares every component with PCU-Select except the learned conditional scorer, which the raw proxy replaces. This makes it the controlled “simple scorer” comparison the main table needs: the PCU-Select–proxy difference measures the marginal contribution of the learned scorer over a forward-only proxy, holding clustering and quota fixed. It requires no

per-PEFT backward pass beyond the $\tilde{\alpha}_p^\omega$ reweighting, so its selection cost matches the LESS-style apply cost without the trained-scorer offline investment.

Reproducibility Supplement

This supplement documents the pipeline configuration read directly from the released implementation. Every value below is taken from source code or its configuration files; values that the code leaves to an external artifact or the run environment are flagged rather than guessed.

Data provenance, deduplication, and contamination.

The candidate pool holds 300K mixed instruction examples loaded from a single JSONL corpus. The released loader enforces *exact-content* deduplication by rejecting duplicate content-hash sample identifiers (hashed over instruction, response, and source), and performs no near-duplicate, n -gram, or embedding-based deduplication. We state two limitations plainly. First, the constituent corpora and their licenses are not enumerated in the released configuration. Second, **we did not run a benchmark-contamination check** between the 300K pool and the four evaluation benchmarks; no decontamination routine exists in the released code. We therefore cannot rule out overlap between pool examples and benchmark test items, and we make no decontamination claim.

Table 5: Per-task normalized improvement of PCU-Select at the 10% budget. Absolute scores (Random, RDS+, LESS, PCU) are seed- and PEFT-averaged on each task’s native metric. ΔRand and $\Delta\text{RDS+}$ are *relative* gains $(100 \cdot (\text{PCU} - b)/b)$, which place the four heterogeneous metrics on a common scale so that cross-task point-differences are not treated as equivalent. ΔLESS is the absolute paired difference in native points against the strongest baseline, with a 95% bootstrap CI over the five PEFT cells: a CI that excludes zero marks a per-task win (GSM8K, MMLU) or loss (TyDiQA), and the sign flips across tasks, which the averaged Table 2 conceals.

Task	Metric	Random	RDS+	LESS	PCU	ΔRand	$\Delta\text{RDS+}$	ΔLESS (95% CI)
GSM8K	EM	33.43	36.08	36.50	37.01	+10.7%	+2.6%	+0.51 [+0.03, +0.98]
HumanEval	Pass@1	29.54	31.57	32.88	32.80	+11.0%	+3.9%	−0.08 [−0.81, +0.58]
MMLU	Acc	44.75	47.59	48.48	49.15	+9.8%	+3.3%	+0.67 [+0.24, +1.08]
TyDiQA	F1	48.98	52.08	53.07	52.30	+6.8%	+0.4%	−0.77 [−1.17, −0.43]

Task sketch construction and HumanEval leakage.

Each task sketch is 32 examples sampled with a fixed seed (0) from a per-task source pool, stratified into response-length terciles (with a plain-random fallback when the pool has fewer than 3×32 items). The intended source is the task’s train or dev split, excluding the test set; the released builder does not enforce this—it samples from whatever pool it is handed and applies no test-set exclusion or overlap check. HumanEval has no official train/dev split. For HumanEval the released pipeline reads the sketch source from a task JSONL but neither constructs a leakage-safe source nor verifies disjointness from the 164-problem evaluation set. We flag this as an unmitigated leakage risk: if the HumanEval sketch draws from HumanEval prompts, it overlaps the evaluation set, and nothing in the code would detect it.

PEFT registry. Table 6 lists all 17 PEFT configurations, partitioned into the five seen configurations the scorer trains on, nine unseen-configuration targets (same families, settings outside the training support), and three out-of-distribution family targets. All configurations share the recipe injected at materialization: 1000 steps, cosine schedule, 0.03 warmup ratio; the learning rate is per configuration.

Intervention sites and gradient signatures. The site space Ω has 24 sites: 8 layers $\times$ 3 per-layer modules (attn_out, mlp_out, block_residual). Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices $\{3, 7, 11, 15, 19, 23, 27, 31\}$. Each site carries a Johnson–Lindenstrauss random projection $\Phi_\omega \in \mathbb{R}^{256 \times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed $\text{SHA256}(\text{site}, \text{global_seed}=0)$; projected gradients are L_2 -normalized per row. A PEFT configuration activates a site through an operator-weight prior ρ by family (additive low-rank 1.0, multiplicative 0.6, additive bottleneck 0.8, prefix 0.4, bias-shift 0.3) scaled by $\tanh(\eta \text{cap})$ with $\eta=1.0$.

Conditional scorer. The scorer has three input towers—sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{192}$, task sketch $z_t \in \mathbb{R}^{848}$ —each a LayerNorm–Linear–GELU–Linear–LayerNorm MLP projecting to hidden size 256 (128 for the PEFT tower). A FiLM modulation and a bilinear fusion ($256 \times 128 \rightarrow 64$) feed a two-head output (mean μ and heteroscedastic σ , with a softplus floor of 10^{-3}). Training is two-phase: Phase A pretrains on the low-fidelity proxy for

3 epochs at $\text{lr } 3 \times 10^{-4}$ with loss weights (rank, reg, proxy, unc) = (0, 0, 1, 0); Phase B jointly finetunes for 2 epochs at $\text{lr } 10^{-4}$ with weights (1.0, 0.3, 0.5, 0.2). Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against high-fidelity labels, a Huber proxy-distillation loss against the low-fidelity labels, and a Gaussian negative-log-likelihood uncertainty loss. The released trainer uses no dropout and no early stopping or explicit validation split; phase loaders are supplied pre-built.

Selection, clustering, and quotas. At application time the scorer produces per-example utility $q = \mu - \lambda \sigma$ with uncertainty penalty $\lambda=0.2$. Semantic clustering uses MiniBatch k -means over joint embeddings with $k = \max(50, \lfloor \sqrt{N} \rfloor)$, batch size 4096, $n_{\text{init}}=3$, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \text{round}(B (v_k^+)^{\alpha} |C_k|^{1-\alpha} / Z)$ with $\alpha=0.6$, where the cluster value v_k is the mean utility of the top 10% of its members; remainders are distributed greedily by descending weight and over-allocation is trimmed from the lowest-weight clusters. Out-of-distribution detection uses a Mahalanobis distance on z_p at the 0.95 quantile; the calibration head is a zero-initialized linear residual fit for 200 epochs at $\text{lr } 5 \times 10^{-3}$ (Adam, MSE).

Multi-fidelity labels. High-fidelity supervision uses 10,000 triples split 0.5/0.3/0.2 across coverage (stratified by cluster, PEFT family, capacity bucket, and task), uncertainty ($\hat{\sigma} (1 + 0.5 \text{ReLU}(\hat{u}_{\text{lo}}))$), and boundary (scorer-vs-true rank disagreement) sampling. Labels are short-horizon loss reductions on the sketch, measured from two frozen anchors— θ_{base} (the pretrained backbone) and θ_{warm} (a rank-8 attention-only LoRA trained for 200 steps at $\text{lr } 10^{-4}$)—at horizons $\{1, 4\}$ with weights $\{0.4, 0.6\}$, rank-normalized within each (PEFT, task, anchor, horizon) bucket.

Cost model and seeds. GPU-hours are computed as (wall seconds $\times$ GPU count)/3600. The offline cost is 11.2 GPU-hours (feature extraction 2.4, low-fidelity labels 2.9, high-fidelity labels 4.5, scorer training 1.4); the per-PEFT influence-recomputation baseline costs 6.8 GPU-hours; break-even is $T = C_{\text{offline}} / (C_{\text{specific}} - C_{\text{apply}})$. The GPU model is not pinned in the cost model, which scales by an environment GPU count. The released pipeline seeds Python, NumPy, PyTorch, and CUDA from a single global

Config	Family	Target modules	Capacity	α	lr	Group
L-r8-qv	LoRA	q, v	$r=8$	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	$r=16$	32	2e-4	seen
L-r8-mlp	LoRA	up, down	$r=8$	16	2e-4	seen
IA3-attnmlp	(IA) ³	k, v, down	–	–	5e-4	seen
AD-b64	Adapter	o, down	$b=64$	–	3e-4	seen
L-r4-qv	LoRA	q, v	$r=4$	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	$r=32$	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	$r=64$	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	$r=8$	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	$r=8$	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	$r=16$	32	5e-4	unseen-config
AD-b16	Adapter	o, down	$b=16$	–	3e-4	unseen-config
AD-b256	Adapter	o, down	$b=256$	–	3e-4	unseen-config
IA3-attnonly	(IA) ³	k, v	–	–	5e-4	unseen-config
PRE-116	Prefix	attention	len=16	–	2e-4	ood-family
PT-132	P-Tuning	attention	len=32	–	2e-4	ood-family
BF	BitFit	biases	–	–	2e-4	ood-family

Table 6: PEFT registry. Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

seed (default 0); reported downstream numbers average three target-training seeds.